

Automated Research for LLM Policy Synthesis in Sequential Social Dilemmas

Víctor Gallego

Komorebi AI Technologies, Madrid, Spain

`victor.gallego@komorebi.ai`

Abstract

We apply the *autoresearch* paradigm to multi-agent LLM policy synthesis. A *researcher agent* $\mathcal{R}$ (Claude Opus 4.6) iteratively modifies the synthesis pipeline—system prompts, feedback construction, helper libraries, iteration logic—under which a *policy-synthesizer* LLM $\mathcal{M}$ generates cooperative strategies for Sequential Social Dilemmas. The researcher never modifies the environment or evaluation; everything else is fair game. We conduct 12 autonomous runs across two games (Cleanup and Gathering), two policy LLMs (Gemini 3.1 Pro and Claude Sonnet 4.6), and two social welfare objectives (utilitarian efficiency and Rawlsian maximin). The researcher reliably exceeds hand-designed baselines: in Cleanup, efficiency improves by 16% (Gemini) and 128% (Sonnet) over the dense feedback configuration of prior work. Most strikingly, maximin-optimized pipelines achieve near-perfect equality ($E=0.98$) with negligible efficiency loss (-1%), discovering duty rotation as the key fairness mechanism—never found under efficiency optimization. The contrast with Gathering, where efficiency optimization alone yields $E>0.94$, reveals that game structure determines whether fairness requires explicit optimization: in provision dilemmas fairness needs designed mechanisms, while in restraint dilemmas it emerges as a byproduct of coordination.

1 Background

We build on the iterative LLM policy synthesis framework of Gallego [1]. An SSD is a partially observable Markov game $\mathcal{G} = \langle N, \mathcal{S}, \{A_i\}_{i=1}^N, T, \{R_i\}_{i=1}^N, H \rangle$. Social outcomes are evaluated via four metrics [3]: efficiency U , equality E , sustainability S , and peace P . We additionally consider the *maximin* (Rawlsian) welfare criterion $\min_i R_i$, which maximizes the return of the worst-off agent.

We study two SSDs that represent complementary dilemma types. **Cleanup** [7] is a *public goods provision* game ($N=10$): a river accumulates waste while an orchard grows apples. Cleaning costs -1 per beam but enables apple regrowth for everyone ($+1$ per apple). The dilemma: free-ride on cleaning vs. contribute. **Gath-**

ering [8, 3] is a *common pool resource* game ($N=4$): agents collect shared apples and may tag rivals to temporarily remove them. The dilemma: over-harvest and aggress vs. cooperate and share. The games differ in their cost structure—asymmetric provision vs. symmetric restraint—a distinction that drives our key findings.

A frozen LLM $\mathcal{M}$ acts as a policy synthesizer. Given a system prompt p and a feedback prompt q_k , it generates a Python policy $\pi_{k+1} = \mathcal{M}(p, q(\pi_k, \mathcal{F}_k^\ell))$, where ℓ specifies the feedback level and $\mathcal{F}_k = (\bar{r}_k, \mathbf{m}_k)$ contains the mean per-agent reward $\bar{r}_k$ and the social metrics vector $\mathbf{m}_k = (U_k, E_k, S_k, P_k)$. All N agents execute the same policy (homogeneous self-play). Gallego [1] compared two hand-designed feedback levels—sparse (reward-only) and dense (reward + social metrics)—and showed that dense feedback consistently improves cooperative outcomes. The dense feedback configuration achieves $U=1.93\pm0.73$ (Gemini) and $U=0.86\pm0.62$ (Sonnet) on Cleanup.

2 Two-Level Framework

We introduce a two-level system where a *researcher agent* $\mathcal{R}$ optimizes the conditions under which the policy synthesizer $\mathcal{M}$ operates. The key insight is that the entire inner-loop codebase—not just the feedback level—is a designable artifact. Figure 1 illustrates the architecture.

2.1 Configuration Space

Let $\mathcal{C}$ denote the space of *pipeline configurations*. Each configuration $c \in \mathcal{C}$ specifies the full inner-loop setup:

$$c = (p, \ell, \phi, \mathcal{H}, \iota, v) \quad (1)$$

where p is the system prompt, ℓ is the feedback template, ϕ is the feedback construction function, $\mathcal{H}$ is the helper function library, ι specifies the iteration logic (number of iterations K , sampling strategy, temperature schedule), and v is the validation pipeline. Table 1 provides concrete examples.

The sparse vs. dense comparison of [1] corresponds to a single binary choice within the ℓ component. Our framework opens the full configuration space to automated search.

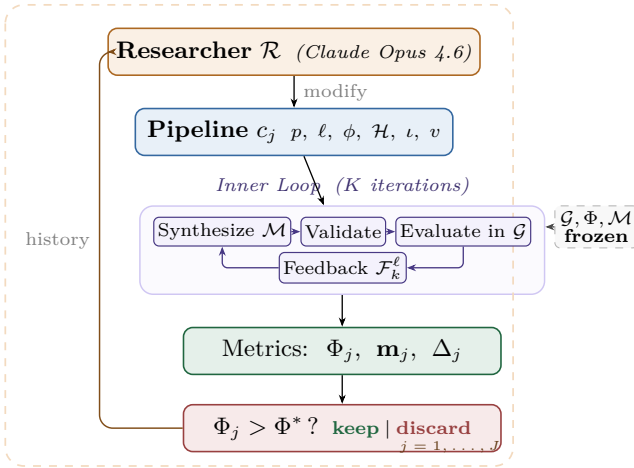


Figure 1: Two-level automated research framework (Algorithm 1). *Outer loop*: the researcher $\mathcal{R}$ modifies the pipeline configuration c_j , runs the inner loop, and keeps or discards based on Φ . *Inner loop*: the policy LLM $\mathcal{M}$ iteratively synthesizes, validates, and evaluates policies in the frozen environment $\mathcal{G}$.

Table 1: Configuration components modifiable by the researcher. The environment simulator, ground-truth evaluation, and policy LLM weights are frozen.

	Scope	Examples
p	System prompt	Hints, worked examples
ℓ	Feedback template	Metric subset, framing
ϕ	Feedback fn	Derived metrics, trends
$\mathcal{H}$	Helper library	Pathfinding, zones
ι	Iteration logic	K , eval seeds, best-of- n
v	Validation	Safety checks, smoke tests

2.2 Inner Loop (Policy Synthesis)

Given a configuration c , the inner loop executes K iterations of LLM policy synthesis:

$$\pi_c^* = \text{INNERLOOP}(\mathcal{M}, \mathcal{G}, c) \quad (2)$$

following Algorithm 1 of [1]: at each iteration k , $\mathcal{M}$ generates a policy from the system prompt p and feedback constructed according to (ℓ, ϕ) , using helpers $\mathcal{H}$. The policy is validated via v and evaluated in N -agent self-play over seeds S .

The inner loop output is evaluated on held-out seeds via a fixed ground-truth objective:

$$\Phi(c) = \text{EVAL}(\pi_c^*; \mathcal{G}, S_{\text{held-out}}) \quad (3)$$

where Φ maps from social outcomes to a scalar. We consider two objectives:

$$\Phi_U = U \quad (\text{utilitarian efficiency}) \quad (4)$$

$$\Phi_{\min} = \min_i R_i \quad (\text{Rawlsian maximin}) \quad (5)$$

Algorithm 1 Two-Level Automated Research

Require: Game $\mathcal{G}$, policy synthesizer $\mathcal{M}$, researcher $\mathcal{R}$, system prompt for researcher $p_{\mathcal{R}}$, initial configuration c_0 , outer iterations J , ground-truth objective Φ , held-out seeds $S_{\text{held-out}}$

Ensure: Best configuration c^* , best policy π^*

```

1:  $\pi_0^* \leftarrow \text{INNERLOOP}(\mathcal{M}, \mathcal{G}, c_0)$ 
2:  $\Phi_0 \leftarrow \text{EVAL}(\pi_0^*; \mathcal{G}, S_{\text{held-out}})$ 
3:  $history \leftarrow \{(c_0, \Phi_0, \mathbf{m}_0, \emptyset)\}$ 
4: for  $j = 1, \dots, J$  do
5:    $c_j \leftarrow \mathcal{R}(p_{\mathcal{R}}, \text{CODE}(c_{j-1}), history)$ 
6:   if  $\neg \text{VALIDATECONFIG}(c_j)$  then  $\text{retry} \leq R$  times
7:      $\pi_j^* \leftarrow \text{INNERLOOP}(\mathcal{M}, \mathcal{G}, c_j)$ 
8:      $\Phi_j \leftarrow \text{EVAL}(\pi_j^*; \mathcal{G}, S_{\text{held-out}})$ 
9:      $\Delta_j \leftarrow \text{DIFF}(c_{j-1}, c_j)$  // code diff
10:     $history \leftarrow history \cup \{(c_j, \Phi_j, \mathbf{m}_j, \Delta_j)\}$ 
11:  end for
12:  $j^* \leftarrow \arg \max_j \Phi_j$ 
13: return  $c_{j^*}, \pi_{j^*}^*$ 

```

2.3 Outer Loop (Automated Research)

The researcher agent $\mathcal{R}$ iteratively modifies the pipeline configuration. Following the autoresearch paradigm [2], $\mathcal{R}$ operates on the inner-loop codebase as a modifiable artifact, proposing changes, observing outcomes, and refining. The procedure is formalized in Algorithm 1.

At each outer iteration j , the researcher $\mathcal{R}$ receives:

- The **full source code** of the current configuration c_{j-1} (prompts, feedback construction, helpers, iteration logic).
- The **experiment history**: for each prior iteration, the code diff Δ_i , ground-truth score Φ_i , and social metrics vector $\mathbf{m}_i$.
- The **environment source code** (read-only), enabling the researcher to reason about game mechanics.

The researcher proposes a new configuration c_j by generating code modifications. This mirrors the autoresearch setup, where an AI agent modifies `train.py` and observes validation loss; here, the researcher modifies the inner-loop codebase and observes social welfare.

2.4 Structural Analogy

Table 2 formalizes the correspondence between autoresearch for LLM pretraining and our two-level system.

The key structural property shared by both systems: the agent modifies *how learning happens* (the pipeline/training code), not the learned artifact directly (the policy/model weights). The output of each outer iteration is a *reusable configuration*—transferable across policy LLMs, games, and random seeds—rather than a single policy.

Table 2: Structural analogy between autoresearch [2] for LLM pretraining and our framework for multi-agent policy synthesis.

	Autoresearch	This work
Modifiable artifact	<code>train.py</code> (model + optimizer)	Inner-loop codebase ($p, \ell, \phi, \mathcal{H}, \iota, v$)
Frozen infrastructure	<code>prepare.py</code> (data, tokenizer, eval)	Environment $\mathcal{G}$, eval Φ , LLM $\mathcal{M}$
Metric	val_bpb $\downarrow$	Φ (social welfare) $\uparrow$
Budget	5 min wall clock	K iters $\times$ $ S $ seeds
Agent	AI coding agent	Researcher $\mathcal{R}$

2.5 Design Principles

Separation of concerns. The researcher $\mathcal{R}$ and the synthesizer $\mathcal{M}$ are distinct LLM calls with distinct roles. $\mathcal{R}$ reasons about *what information and tools help $\mathcal{M}$ write better policies*; $\mathcal{M}$ reasons about *what policy to write given its current context*. Using different underlying models ($\mathcal{R} \neq \mathcal{M}$) avoids conflation and enables cross-model experiments.

Read-only environment. The researcher can read the environment source code to understand game mechanics but cannot modify the simulator. This prevents the trivial solution of making the game easier and cleanly separates mechanism design from environment design.

Fixed compute budget. Each inner loop runs for exactly K iterations with $|S|$ seeds. The researcher cannot inflate performance by running more iterations, but can change what happens *within* those K iterations (e.g., best-of- n sampling, varying the prompt structure mid-run).

Diff-based history. The researcher sees code diffs Δ_j from previous experiments alongside their outcomes. This enables causal reasoning about which modifications drove which improvements, mirroring autoresearch’s experiment logs.

3 Connection to Automated Mechanism Design

The two-level structure admits a mechanism design interpretation. The researcher $\mathcal{R}$ acts as a *mechanism designer*: it controls the information structure (what metrics to reveal, how to frame them), the action space (what helper functions are available), and the incentive structure (how feedback is presented) under which the policy synthesizer $\mathcal{M}$ operates. The synthesizer acts as the *agent* within the designed mechanism.

This connects to the automated mechanism design literature [4], where a principal designs rules to induce desired behavior from self-interested agents. In our setting:

- The **principal** is the researcher $\mathcal{R}$, optimizing the ground-truth objective Φ .
- The **agent** is the synthesizer $\mathcal{M}$, optimizing per-agent reward as instructed.
- The **mechanism** is the configuration c : prompts, feedback, helpers, iteration logic.
- The **outcome** is the social welfare of the resulting multi-agent policy π_c^* .

A key distinction from classical mechanism design: $\mathcal{M}$ is not *strategically* self-interested—it follows instructions—but it has *bounded rationality* in the sense that its ability to synthesize effective policies depends on the information and tools provided. The researcher’s task is thus closer to *information design* [5]: choosing what to reveal to help $\mathcal{M}$ navigate the cooperation–defection tension. Our experiments (Section 4) show that the researcher designs qualitatively different information structures depending on the welfare objective Φ , supporting this interpretation empirically.

4 Experiments

4.1 Setup

We conduct 12 autonomous researcher runs across a factorial design. The researcher agent $\mathcal{R}$ is Claude Opus 4.6 in all runs. Each run operates on a dedicated git branch, modifying only the pipeline files (`pipeline/`).

Design. For Cleanup: 2 policy LLMs $\times$ 2 objectives $\times$ 2 replications = 8 runs. For Gathering: 2 policy LLMs $\times$ 1 objective $\times$ 2 replications = 4 runs. Maximin runs are unnecessary for Gathering because efficiency optimization alone achieves equality $E > 0.94$ (the game’s symmetric cost structure makes fairness a free byproduct of coordination).

Models. Policy synthesizer $\mathcal{M}$: Gemini 3.1 Pro (Google) or Claude Sonnet 4.6 (Anthropic). Both use extended thinking. These are the same models evaluated in [1].

Baselines. The hand-designed dense feedback (reward + social metrics) configuration from [1] serves as the initial pipeline c_0 for all runs. On Cleanup ($N=10$), this baseline achieves $U=1.93\pm 0.73$ with Gemini and $U=0.86\pm 0.62$ with Sonnet (mean $\pm$ s.d. across 4 runs each).

Inner loop parameters. $K=3$ synthesis iterations (or $K=2$ if the researcher elects), evaluated over $|S|=5\text{--}12$ random seeds (researcher-configurable). The researcher

Table 3: Autoresearch results. Mean $\pm$ s.d. across runs per condition (2 runs for autoresearch, 4 for Cleanup baselines, 2 for Gathering baselines). Baseline: unmodified pipeline with dense feedback from [1].

Policy LLM	Target	U	E	$\min_i R_i$
<i>Cleanup (N=10) — Baseline</i>				
Gemini 3.1 Pro	—	1.93 $\pm$ 0.73	0.17 $\pm$ 0.52	—
Sonnet 4.6	—	0.86 $\pm$ 0.62	−0.02 $\pm$ 0.36	—
<i>Cleanup (N=10) — Autoresearch</i>				
Gemini 3.1 Pro	Φ_U	3.20 $\pm$ 0.05	0.55 $\pm$ 0.06	—
	$\Phi_{\min}$	3.16 $\pm$ 0.04	0.98 $\pm$ 0.00	290 $\pm$ 6
Sonnet 4.6	Φ_U	3.12 $\pm$ 0.02	0.66 $\pm$ 0.03	—
	$\Phi_{\min}$	2.57 $\pm$ 0.36	0.91 $\pm$ 0.07	179 $\pm$ 20
<i>Gathering (N=4) — Baseline</i>				
Gemini 3.1 Pro	—	2.04 $\pm$ 0.54	0.90 $\pm$ 0.11	412 $\pm$ 224
Sonnet 4.6	—	0.03 $\pm$ 0.00	0.54 $\pm$ 0.00	0 $\pm$ 0
<i>Gathering (N=4) — Autoresearch</i>				
Gemini 3.1 Pro	Φ_U	2.49 $\pm$ 0.02	0.98 $\pm$ 0.00	582 $\pm$ 1
Sonnet 4.6	Φ_U	2.52 $\pm$ 0.01	0.96 $\pm$ 0.02	576 $\pm$ 20

ran $J=3$ –17 outer iterations per run (mean: 8.3), with a keep rate of 18%–100% (mean: 63%).

4.2 Results

Table 3 presents the main results, showing the best configuration achieved by the researcher in each condition, compared against the hand-designed baseline of [1].

Finding 1: The researcher reliably improves over hand-designed baselines. All 12 runs show substantial improvement regardless of starting point (Figure 2). In Cleanup, the researcher exceeds the unmodified pipeline baseline by 66% with Gemini (U : 1.93 $\rightarrow$ 3.20) and 263% with Sonnet (U : 0.86 $\rightarrow$ 3.12). Strikingly, the Sonnet improvement nearly closes the gap with Gemini ($U=3.12$ vs. 3.20), despite a $2\times$ baseline disadvantage (0.86 vs. 1.93). Final efficiency values cluster tightly around $U \approx 3.1$ –3.2 despite baselines varying from 0.29 to 2.70 across individual runs, suggesting the researcher reliably discovers the performance ceiling of each policy LLM.

Finding 2: No efficiency–fairness tradeoff in Cleanup (Gemini). Maximin-optimized Gemini pipelines sacrifice only 1% efficiency (U : 3.16 vs. 3.20) while achieving near-perfect equality (E : 0.98 vs. 0.55) and transforming maximin from deeply negative baselines (−99 to −84) to $\min_i R_i = 290$. The researcher discovers that *fair duty rotation*—time-based cycling of cleaning responsibilities using `agent_id` and `env._step_count`—simultaneously improves worst-off welfare *and* collective output. Because cleaning is a public good, distributing

Table 4: Strategies discovered by the researcher in Cleanup across 4 efficiency-optimized and 4 maximin-optimized independent runs.

Strategy	Φ_U (4 runs)	$\Phi_{\min}$ (4 runs)
Waste-counting helpers	4/4	4/4
Zone/lane partitioning	3/4	4/4
Anti-regression feedback	3/4	2/4
Worked policy examples	2/4	3/4
Cleaning cost economics	2/4	3/4
Time-based role rotation	0/4	4/4

the cleaning cost fairly ensures enough cleaners to sustain apple production.

For Sonnet, there is a moderate tradeoff: efficiency drops from 3.12 to 2.57 under maximin optimization. The gap reflects Sonnet’s harder time implementing complex coordination mechanisms (role rotation, zone assignment) from strategic hints alone.

Finding 3: Game structure determines whether fairness requires explicit optimization. In Cleanup, where cleaning costs are borne asymmetrically (cleaners pay −1, free-riders collect apples), baseline equality ranges from $E=0.04$ to 0.62, and maximin optimization is required to reach $E > 0.9$. In Gathering, where all agents face a symmetric landscape, efficiency optimization alone achieves $E > 0.94$ across all 4 runs—no separate maximin runs are needed.

This generalizes: in *provision* dilemmas with asymmetric costs, fairness requires designed mechanisms (role rotation, duty sharing); in *restraint* dilemmas with symmetric costs, fairness emerges as a free byproduct of efficient coordination. The researcher independently discovers this—it creates role differentiation pipelines only for Cleanup, and pure spatial-coordination pipelines for Gathering.

Finding 4: Convergent discovery of qualitatively different strategies per objective. Despite fully independent runs, the researcher converges on the same core strategies within each condition (Table 4). In Cleanup, waste-counting helpers and spatial zone partitioning appear across all 8 runs. The key qualitative difference: *role rotation*—time-based cycling of cleaning duty—appears in 4/4 maximin runs but 0/4 efficiency runs. Under efficiency optimization, the researcher assigns static cleaning roles (some agents always clean), achieving high collective output at the cost of equality. Under maximin optimization, it discovers rotation as a fairness mechanism.

In Gathering, the researcher discovers BFS-based Voronoi territory partitioning and respawn-timer awareness, with no role differentiation needed—the entire optimization is spatial (who collects which apples) and temporal (respawn-aware positioning).

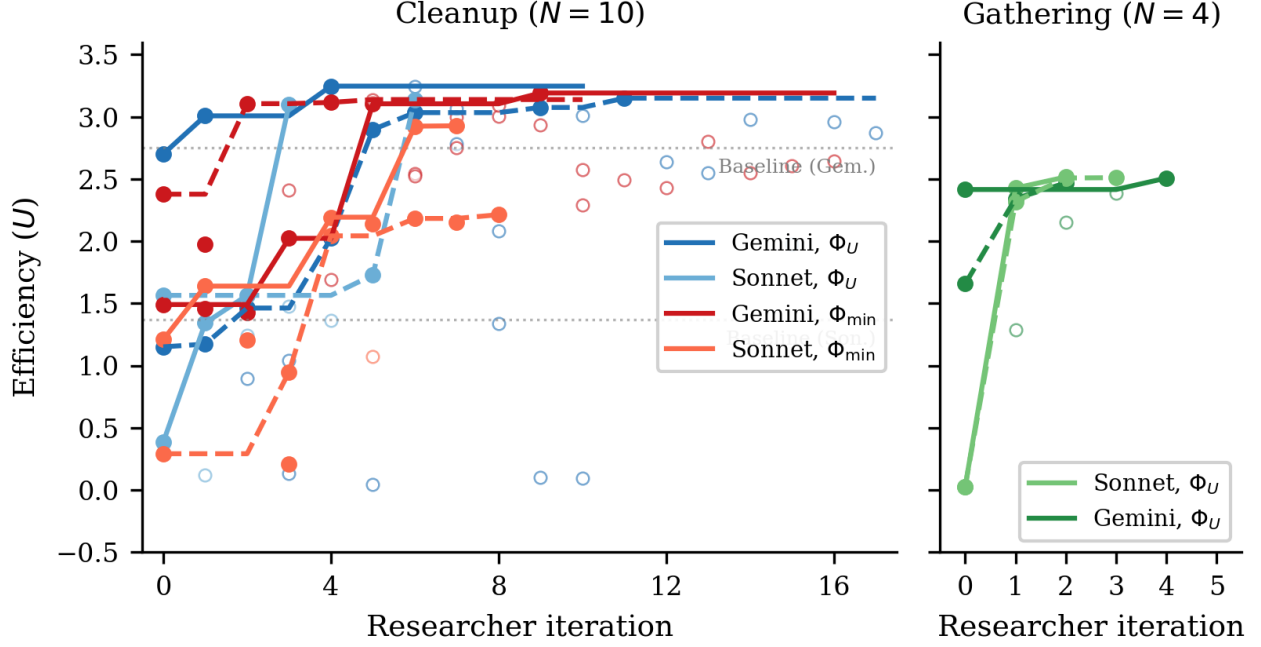


Figure 2: Efficiency (U) across researcher iterations for all 12 runs. *Left*: Cleanup ($N=10$) with 8 runs across 2 LLMs $\times$ 2 objectives (solid lines = kept experiments, open circles = discarded). Dashed horizontal line: hand-designed baseline from [1]. All runs converge to $U \approx 3.1$ – 3.2 despite diverse starting points. *Right*: Gathering ($N=4$) with 4 runs (2 per LLM). All converge to $U \approx 2.5$ within 2–4 iterations despite baselines ranging from 0.03 to 2.42.

Common failure modes. Several patterns led to discarded experiments across all runs: (1) *over-prescription*—adding too many strategic hints confuses the policy LLM, causing generation failures; (2) *iteration regression*— $K=4$ or $K=5$ inner iterations often cause catastrophic regression as the LLM “over-refines” a working policy; (3) *feedback overload*—dense per-agent diagnostics cause over-correction rather than gradual improvement; (4) *variance exposure*—identical configurations can yield $U=3.2$ then $U=0.09$ on different seeds, requiring multi-seed evaluation for stable signals.

5 Discussion and Conclusion

We have presented a two-level framework where a researcher agent autonomously optimizes the synthesis pipeline for multi-agent LLM policies in social dilemmas. Across 12 independent runs, the researcher reliably discovers configurations that exceed hand-designed baselines, and converges on qualitatively similar strategies within each game-objective condition.

Mechanism design in action. Our results empirically support the mechanism design interpretation of Section 3. The researcher acts as an information designer: under Φ_U , it reveals efficiency-oriented information (waste counts, zone assignments) that guides $\mathcal{M}$ toward productive but potentially unequal role allocation. Under $\Phi_{\min}$,

it additionally reveals fairness-oriented structure (rotation schedules, per-agent equity feedback) that guides $\mathcal{M}$ toward egalitarian coordination. The striking absence of a fairness–efficiency tradeoff with Gemini suggests that in public goods provision, the mechanism design problem has a cooperative optimum where both objectives align—the researcher discovers this independently.

Two types of dilemma. The Cleanup–Gathering contrast reveals a structural insight about social dilemmas: *provision* dilemmas (asymmetric costs, one agent’s contribution benefits all) require explicit fairness mechanisms, while *restraint* dilemmas (symmetric costs, each agent faces the same temptation) achieve fairness naturally when coordination improves. This distinction, well-known in the common-pool resource literature, is rediscovered here by an autonomous AI agent—suggesting it is a robust structural feature of these environments.

Limitations. Our experiments use only 2 replications per condition, yielding high uncertainty on some estimates (e.g., Sonnet maximin: $U=2.57 \pm 0.36$). The two SSDs are small gridworld environments; scaling to larger, more complex games remains open. The researcher runs are not fully controlled—the number of outer iterations varies from 3 to 17 (mean 8.3)—making direct comparison across conditions approximate. We do not compare against automated prompt optimization baselines such

as GEPA [6], which optimizes the system prompt rather than the full pipeline; this comparison is an important direction.

Future work. Four directions emerge: (1) *ablation*—freeze all configuration components except one to isolate the relative importance of prompt, feedback, helpers, and iteration logic; (2) *adversarial objectives*—set $\Phi = R_0$ (agent 0’s reward only) to test whether the researcher discovers exploitative pipeline configurations, connecting to the reward hacking analysis of [1]; (3) *cross-game transfer*—run the best Cleanup configuration on Gathering and vice versa, testing game-generalizability; (4) *heterogeneous policies*—extend to settings where agents run different code, enabling richer role specialization. Code is available at <https://github.com/vicgalle/llm-policies-social-dilemmas>.

References

- [1] Víctor Gallego. 2026. Cooperation and Exploitation in LLM Policy Synthesis for Sequential Social Dilemmas. In *Proc. Adaptive and Learning Agents Workshop (ALA 2026)*.
- [2] Andrej Karpathy. 2026. autoresearch: AI agents running research on single-GPU nanochat training automatically. <https://github.com/karpathy/autoresearch>.
- [3] Julien Perolat, Joel Z Leibo, Vinicius Zambaldi, Charles Beattie, Karl Tuyls, and Thore Graepel. 2017. A multi-agent reinforcement learning model of common-pool resource appropriation. *Advances in Neural Information Processing Systems* 30.
- [4] Vincent Conitzer and Tuomas Sandholm. 2002. Complexity of Mechanism Design. In *Proc. 18th Conference on Uncertainty in Artificial Intelligence*. 103–110.
- [5] Emir Kamenica and Matthew Gentzkow. 2011. Bayesian Persuasion. *American Economic Review* 101, 6 (2011), 2590–2615.
- [6] Lakshya A Agrawal et al. 2026. GEPA: Reflective Prompt Evolution Can Outperform Reinforcement Learning. *arXiv:2507.19457*.
- [7] Edward Hughes, Joel Z Leibo, Matthew Phillips, Karl Tuyls, Edgar A Dueñez-Guzmán, Antonio García Castañeda, Iain Dunning, Tina Zhu, Kevin R McKee, R. Koster, Heather Roff, and Thore Graepel. 2018. Inequity aversion improves cooperation in intertemporal social dilemmas. In *Neural Information Processing Systems*.
- [8] Joel Z Leibo, Vinicius Zambaldi, Marc Lanctot, Janusz Janssen, and Thore Graepel. 2017. Multi-agent reinforcement learning in sequential social

dilemmas. In *Proc. 16th Conference on Autonomous Agents and MultiAgent Systems*. 464–473.

A Additional Results

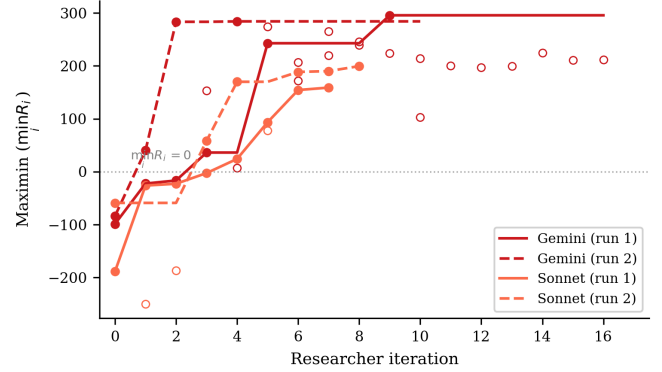


Figure 3: Maximin ($\min_i R_i$) across researcher iterations for the 4 Cleanup maximin-optimized runs. All runs transform deeply negative baselines (worst-off agents losing reward) into substantially positive values. Gemini reaches ~ 290 while Sonnet reaches ~ 160 – 200 . The dashed line marks $\min_i R_i = 0$ (no agent loses reward overall).

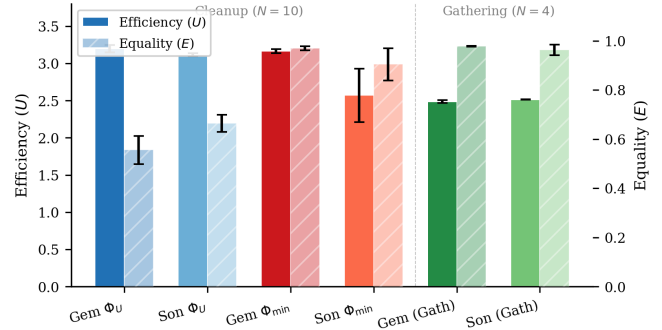


Figure 4: Final efficiency (U , bars) and equality (E , hatched bars) for all 12 runs, grouped by condition. Error bars show half-range across 2 runs. Maximin-optimized Gemini runs achieve both high efficiency and near-perfect equality, while efficiency-optimized runs show moderate equality. Gathering runs achieve high equality regardless of objective.